(swap) Selection Sort $O(n^2)$

	8	2	0	3	6	4	1
	0	2	8	3	6	4	1
	0	1	8	3	6	4	2
	0	1	2	3	6	4	8
	0	1	2	3	6	9	8
	0	1	2	3	4	6	8

i, j, minIdx

arr[i] < arr[minIdx]

0 < 8

(shift) Insertion Sort $O(n^2)$

(UNO)

8	2	1	9	0
2	8	1	9	0
1	2	8	9	0
0	1	2	8	9

temp = 8

Shift operation

temp = 0

i = 1
j = i-1

5	2	8	1	0	3
0	1	2	3	4	5

temp, j, j-1

5	2	8	1	0	3
0	1	2	3	4	5

j, j

temp = 0

Insertion sort logic for shifting

temp = arr[i]

while (j >= 0 && arr[j] > temp)

[ ] → [5]

a[j] a[j+1]

arr[j+1] = arr[j]

j--

* Non-comparison algos :→ ① Count Sort Algorithm

* Single digit whole nos : [0 - 9]

arr | 3 | 4 | 6 | 2 | 4 | 3 | 2 | 3 | 1 | L ← R

Step 1 : Find the max 6 O(n+max)

Step 2 : Create a count array [0-6]

Step 3 : Calculate the frequency of each element of the array

Step 4 : Calculate the cumulative frequency

Step 5 : Create the o/p array

count

0	2	1	3	2	0	1
0	1	2	3	4	5	6

cumulative count

0	2	3	6	8	8	9
0	1	2	3	4	5	6

L-R

o/p

1	1	2	3	3	3	4	4	6
0	1	2	3	4	5	6	7	8

step 6 : Start from the end of the i/p array

O (n + max)

(i) Mx = 6
(ii) Count array 0-6
(iii) Freq
(iv) Cumulative freq
(v) o/p
(vi) end of array

arr[i]

1	3	2	3	4	1	6	4	3

count

0	2	1	3	2	0	1
0	1	2	3	4	5	6

cumulative count

0	2	3	6	8	8	9
0	1	2	3	4	5	6

L-R

1	1	2	3	3	3	4	4	6
0	1	2	3	4	5	6	7	8

L ← R

Merge | Quick | Radix (NCA) [O(n+max)]

(Radix Sort) (Bucket Sort)

* Multi digit nos
* Constant length strings

pavan samar nikhil

5 5 5

(i) Find the max : 427

digits = 3 (passes)

(ii) Create (0-9) 10 buckets

1's, 10's, 100's

(29875) passes = 5

(P1, P2, P3 / countSort)

array

325	427	002	098	001	

P1

001	002	325	427	098	

P2

001	002	325	427	098	

P3

001	002	098	325	427	

0	1	2	3	4	5	6	7	8	9
	001	002					427	098	
	001	325							
	001	427							
	001								
	098								

325 427 325 427 098

Interview Question from Radix Sort Algorithm :

How are the number of passes in radix sort controlled ? {Capgemini, HCL, Mphasis, TCS, Accenture, L&T soft, Tech Mahindra}

How ?

Ans :→ The no of passes in radix sort are dependant on the no of digits in the max value.

pseudo code :→

max = 427

427/1 = 427
427/10 = 42
427/100 = 4
427/1000 = 0

for (int exp = 1 ; max / exp > 0 ; exp *= 10) {

1, 2, 3 slots

countSort (arr, n, exp);